W4

HTML 網頁結構解析與資料擷取

DOM 結構與元素定位 · BeautifulSoup · 多筆擷取與檔案下載

吳彥霖 博士

Python 爬蟲實務應用 · 第 4 週 · 10/06

三節課，三件事

節 1

HTML 結構與元素定位

- DOM 樹：tag / class / id
- DevTools 元素面板定位
- CSS selector

節 2

BeautifulSoup 擷取資料

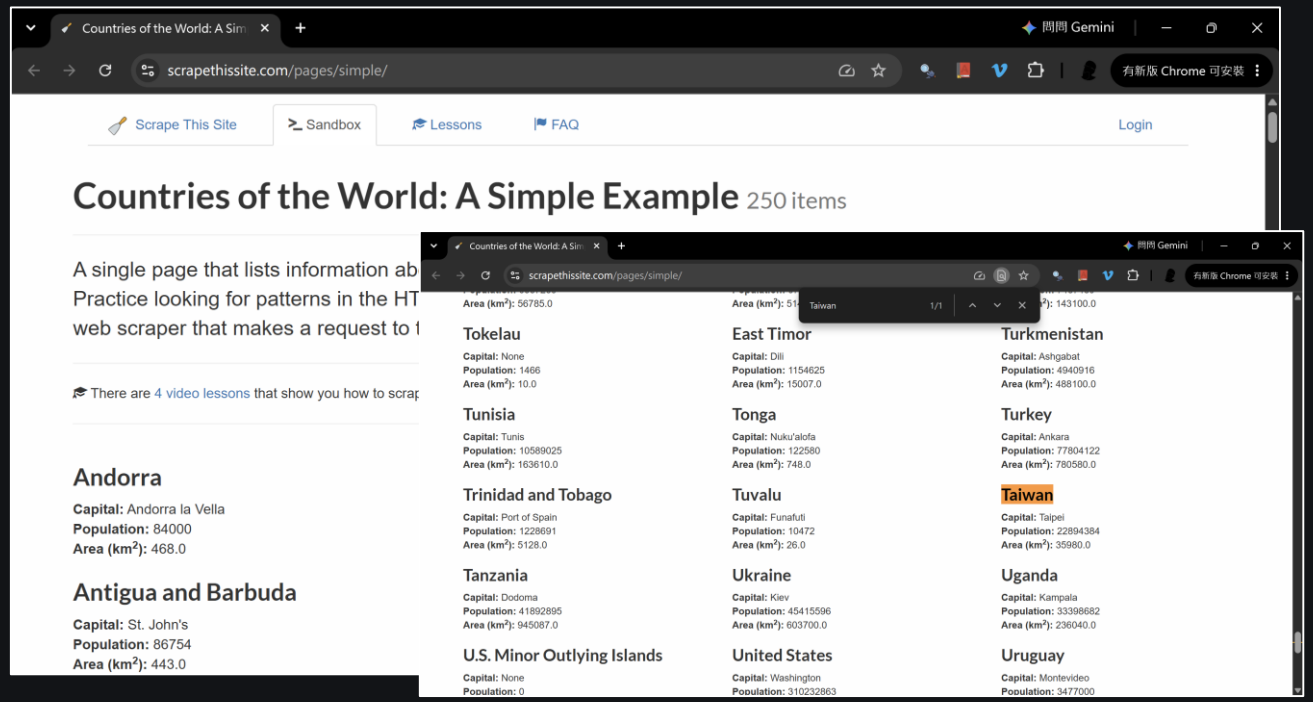
- select 與 find 系列方法
- get_text / 取屬性
- read_html vs BeautifulSoup

節 3

多筆擷取、存檔與下載

- list[dict] → CSV

課堂練習 - 1



https://www.scrapethissite.com/pages/simple/

- 爬取 ScrapeThisSite 練習網站的 Taiwan 相關資訊

Step 1 : 頗析伺服器回應的 Response

```
import requests

URL = "....."
HEADERS = {"User-Agent": "....."}

resp = requests.get(URL, headers=HEADERS,  timeout=10)

# ---------- Response（伺服器回應內容） ----------
print(f"[Response] 狀態碼={resp.status_code} {resp.reason}")
print(f"[Response] Content-Type={resp.headers.get('Content-Type')}")
print(f"[Response] 編碼={resp.encoding}")
print(f"[Response] 花費秒數={resp.elapsed.total_seconds():.2f}")
print(f"[Response] User-Agent={resp.request.headers['User-Agent']}")
```

Step 2 : 取得且解析網頁原始碼

Step 3 : 轉換網頁表格成 DataFrame 清單

Step 4 : DataFrame 寫入 Excel

為什麼爬取不到資料？

網頁	資料長什麼樣子	pd.read_html	該用什麼
Hockey Teams	<table> 表格	可以	read_html 一行搞定
Countries of the World	卡片，一國一張	No tables found	BeautifulSoup：自己定位每個欄位

> **問題**　真實網頁的資料多半「長得像表格，卻不是 <table>」→ 要先看懂 HTML 結構，才能把資料拿出來

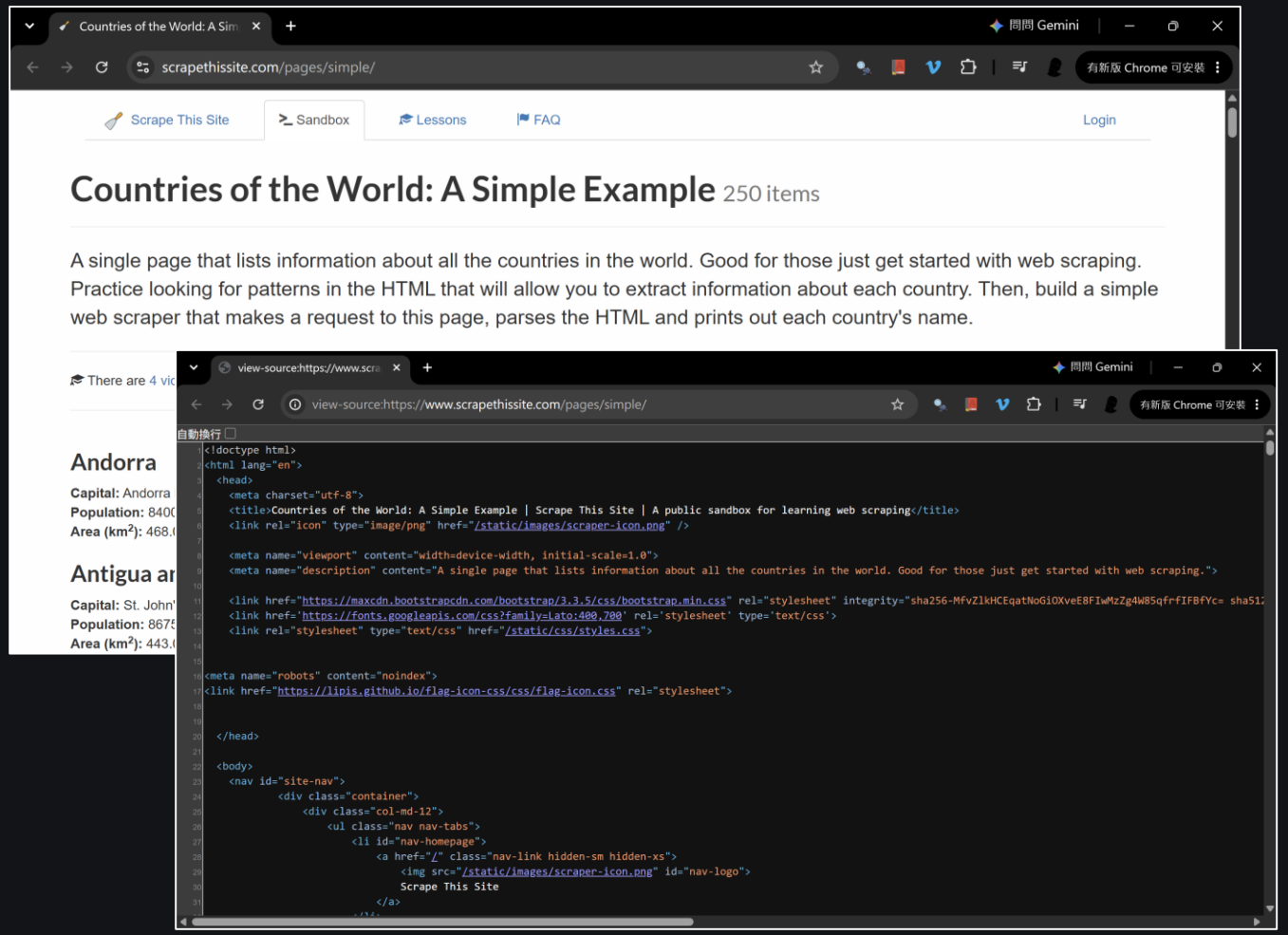
參考文件：

- pandas.read_html

節 1

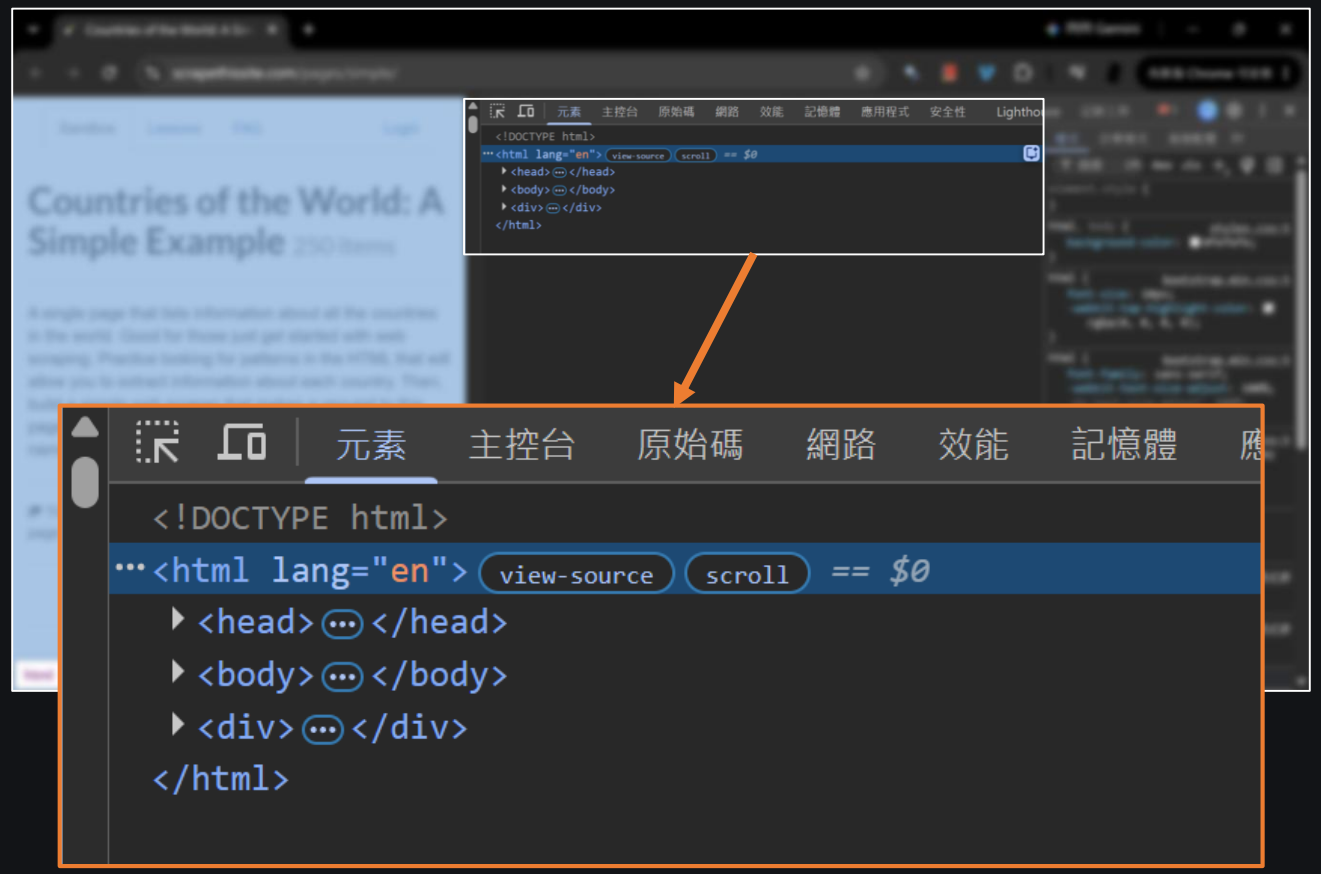
HTML 結構與元素定位

Document Object Model



- 當瀏覽器載入一個 HTML 網頁時，會將 HTML 程式碼解析，且在記憶體中建立一個**樹狀結構**，這個結構稱為 **DOM 樹 (DOM Tree)**。
- DOM 樹為網頁物件化地圖，將網頁上的每一個元素、文字、甚至屬性，皆變成一個個可被 JavaScript 操控的物件(Node)。

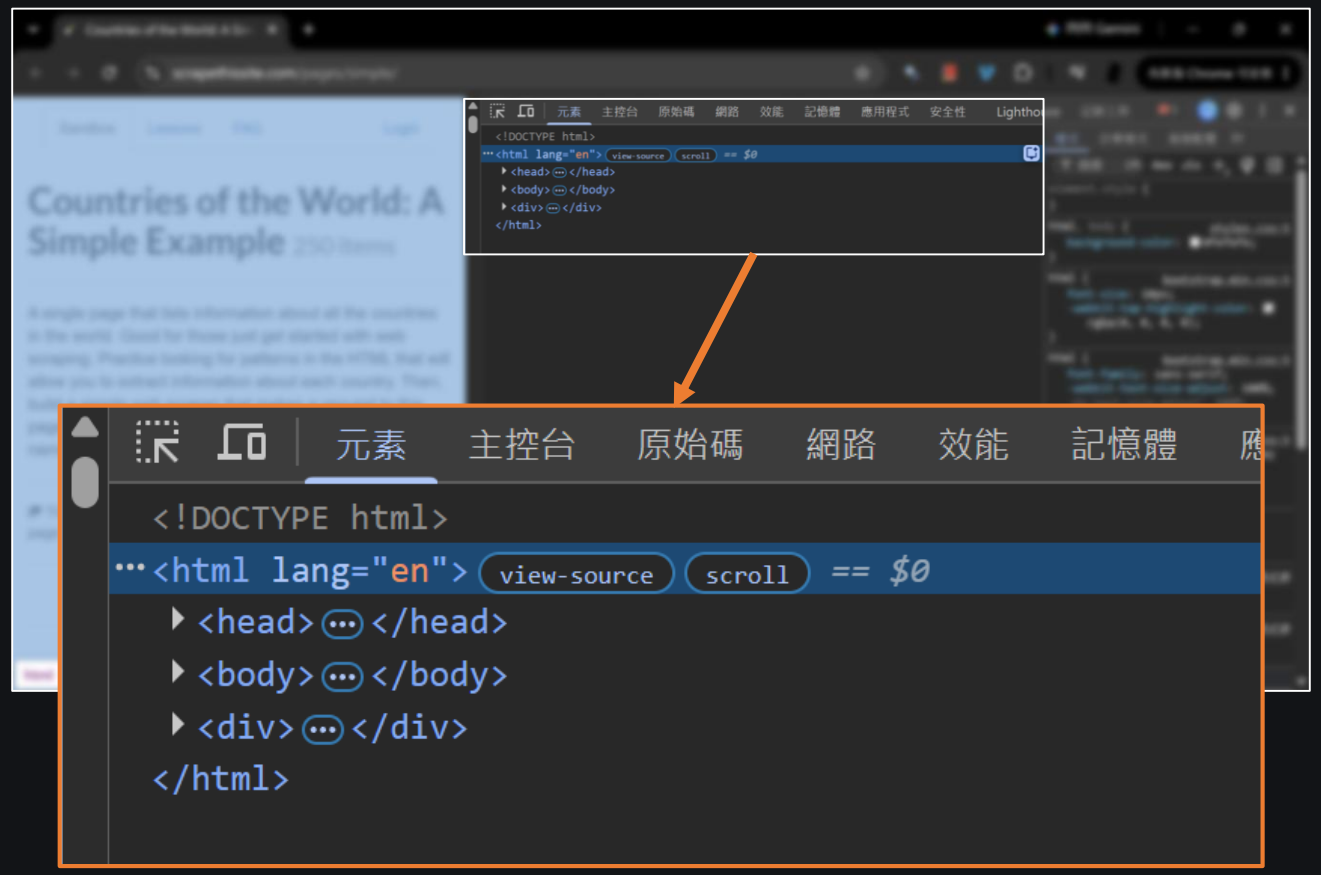
Document Object Model



- **DOM 結構**

```
document                                  ← 根節點（soup）
├── <!DOCTYPE html>                       ← 宣告 HTML 文件
└── html                                  ← 根目錄
    ├── head
    │   ├── meta  charset="utf-8"
    │   └── title
    │       └── "Countries of the World..."
    └── body
        └── section  id= "countries"
            └── div  class= "col-md-4 country"
                ├── h3  class= "country-name"
                │   └── "Andorra"
                └── div  class="country-info"
                    └── span  class="country-capital"
                        └── "Andorra la Vella"
```

Document Object Model

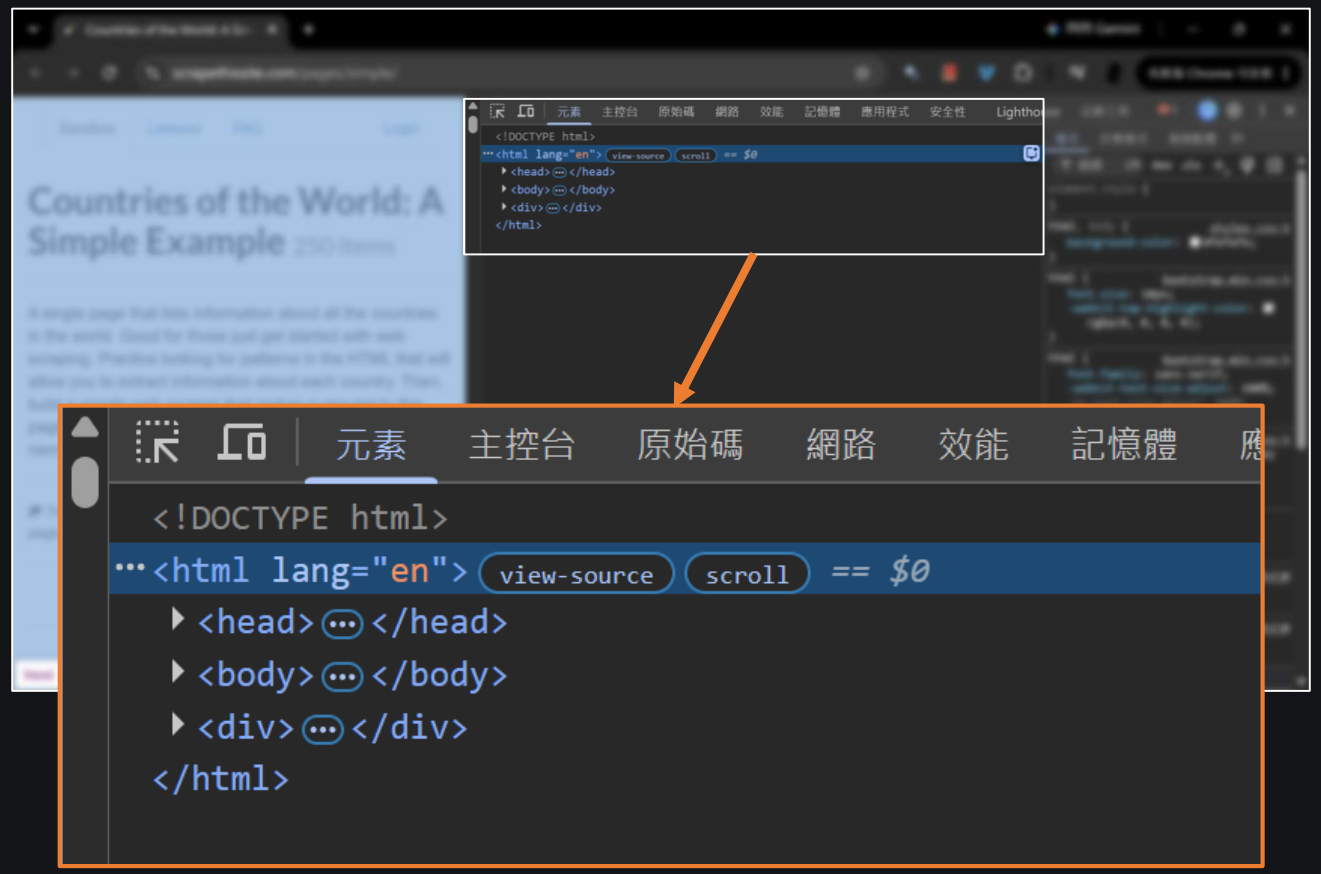


- **DOM 結構**

```
document                          ← 根節點（soup）
├── <!DOCTYPE html>               ← 宣告 HTML 文件
└── html                          ← 根目錄
    ├── head
    │   ├── meta  charset="utf-8"
    │   └── title
    │       └── "Countries of the World..."
    └── body
```

網頁的大腦，放置給瀏覽器與搜尋引擎檢視的設定，
例如：網頁標題 <title>
　　編碼 <meta charset="utf-8">

Document Object Model

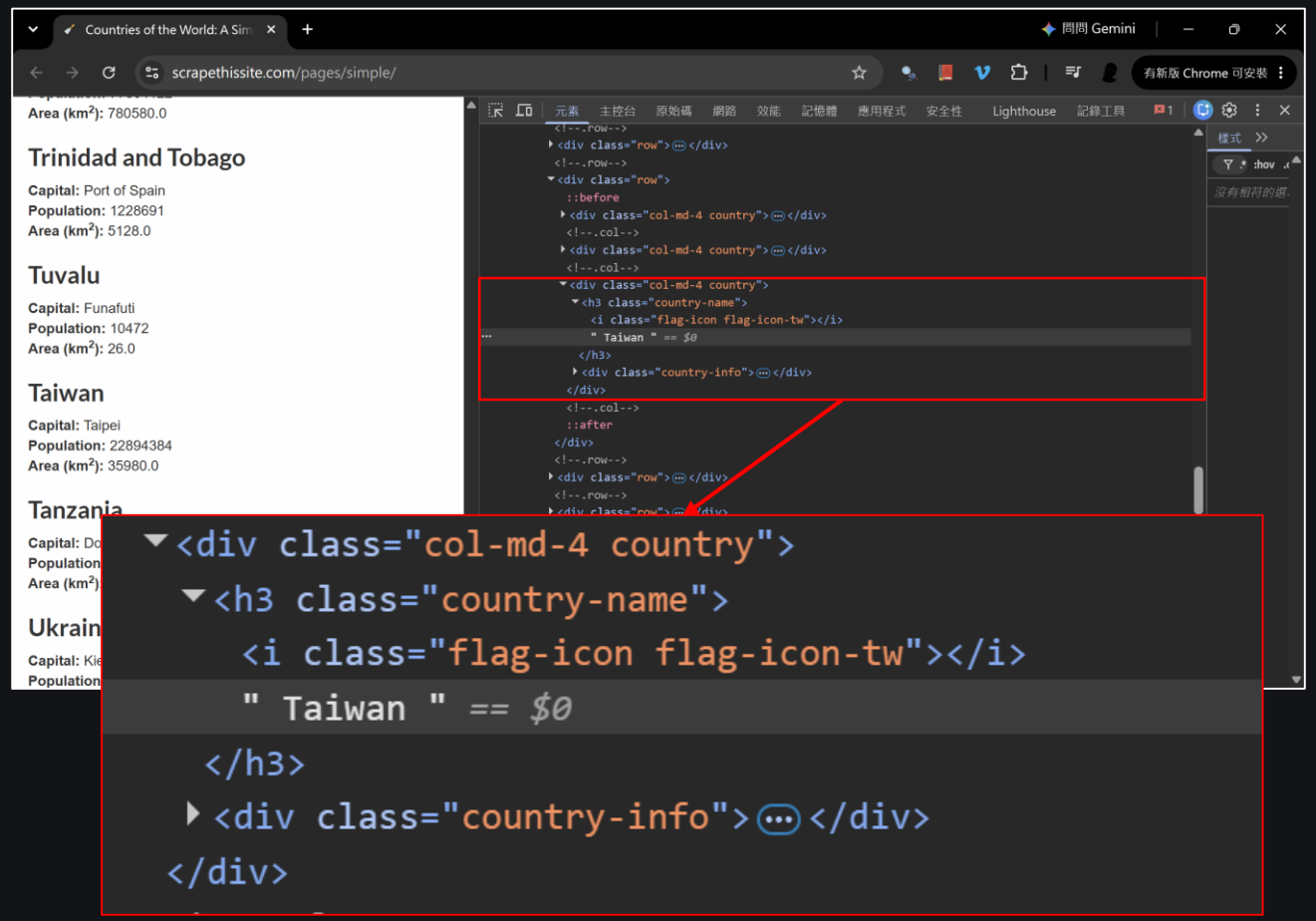


- **DOM 結構**

```
document                                  ← 根節點（soup）
├── <!DOCTYPE html>                       ← 宣告 HTML 文件
└── html                                  ← 根目錄
    ├── head
    │   ├── meta  charset="utf-8"
    │   └── title
    │       └── "Countries of the World..."
    └── body
        └── section  id= "countries"
            └── div  class= "col-md-4 country"
                ├── h3  class= "country-name"
                │   └── "Taipei"
                └── div  class="country-info"
                    └── span  class="country-capital"
                        └── "Andorra la Vella"
```

網頁的身體，用來放置使用者實際在頁面上能看見的文字、圖片或互動內容。

一個元素的組成



一個國家 = 一張卡片

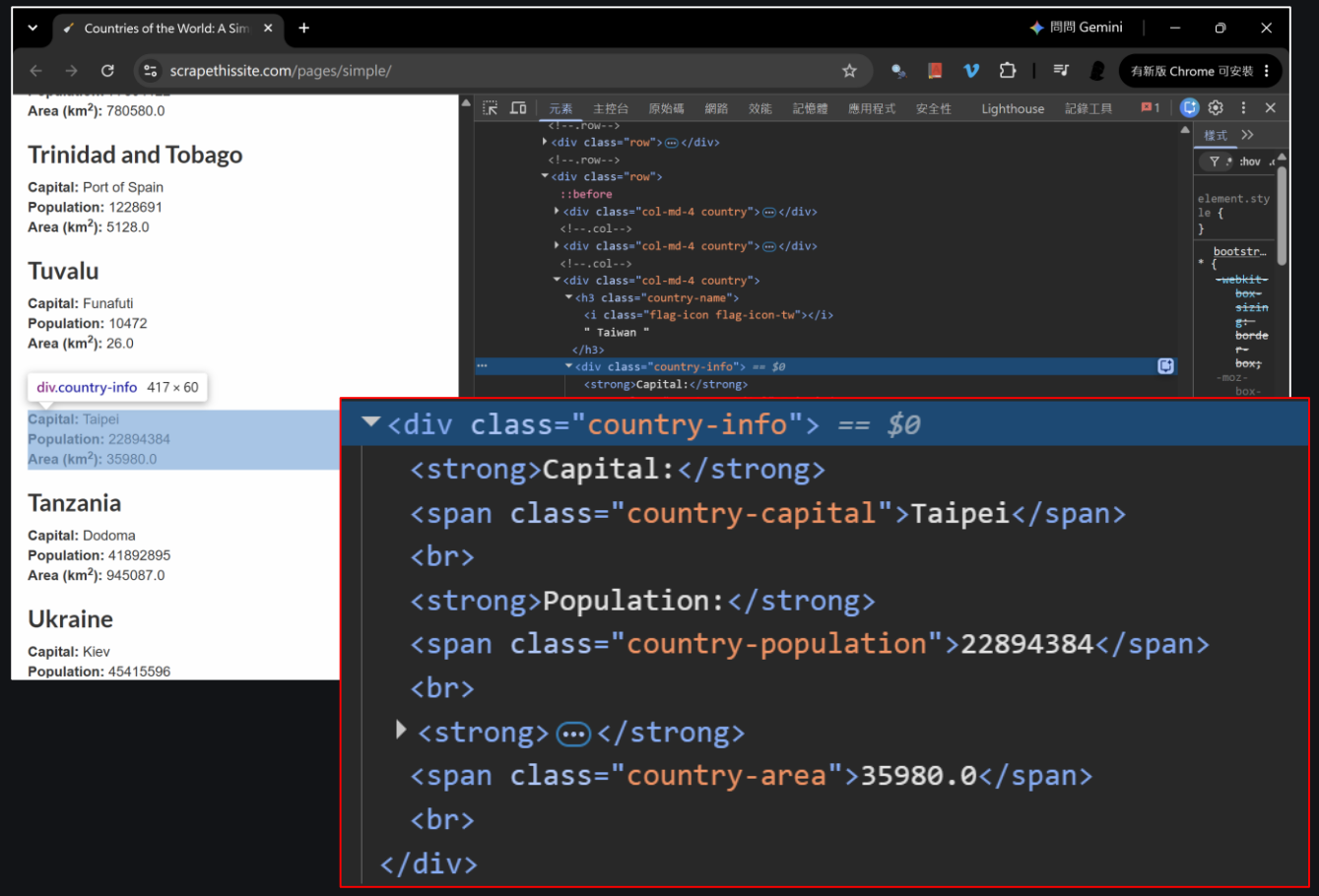
- 元素 = 起始標籤 + 屬性 + 內容 + 結束標籤

h3　class　Taiwan　/h3

定位線索

- ➢ div 區塊、分隔容器
- ➢ h3 第三級標題標籤
- ➢ i 文字字體

一個元素的組成



首都、人口、面積資訊

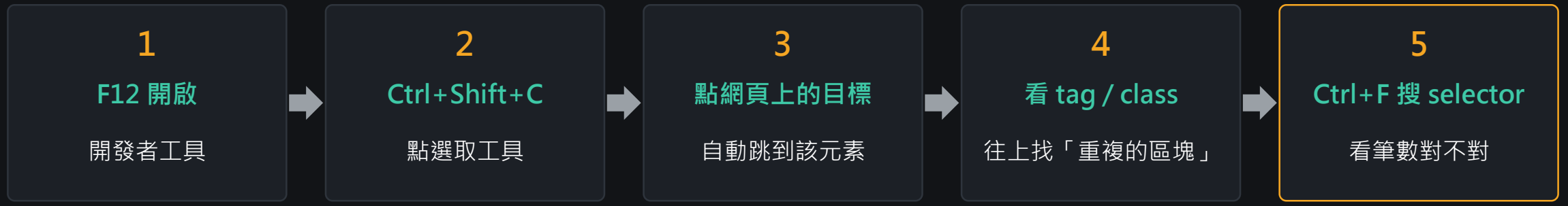
- 元素 = 起始標籤 + 屬性 + 內容 + 結束標籤

h3 class Taiwan /h3

定位線索

- ➢ div 區塊、分隔容器
- ➢ h3 第三級標題標籤
- ➢ i 文字字體

用開發者工具定位元素



結論　先在瀏覽器驗證 selector，再撰寫 Python 程式，可省下大量「為什麼抓不到」的除錯時間。

課堂練習 - 2

在 Countries of the World 中尋找出 5 個欄位的 CSS selector，填進 week_04_exercise_2.py 執行驗證。

week_04_exercise_1.py

```
SELECTORS = {
    "國家卡片": "div.country",
    "國名": ".country-name",
    "首都": ".country-capital",
    "人口": ".country-population",
    "面積": ".country-area",
}
# 程式會自動檢查：每個欄位都要抓到 250 筆
```

預期輸出

```
[OK] 國家卡片  div.country       抓到 250 筆
[OK] 國名    .country-name      抓到 250 筆
[OK] 首都    .country-capital   抓到 250 筆
[OK] 人口    .country-population 抓到 250 筆
[OK] 面積    .country-area      抓到 250 筆
```

節 2

BeautifulSoup 擷取資料

BeautifulSoup：把 HTML 字串變成可搜尋的樹



> **分工**　requests 負責「拿」，BeautifulSoup 負責「找」。

參考文件：

- Beautiful Soup Documentation（Crummy.com）

BeautifulSoup 模組解析網頁

HTML 字串轉成 DOM 樹

- 解析器 Parser

寫法	需要安裝	速度	容錯能力	適合
"html.parser"	不用(Python 內建)	中	良好	**輕量爬蟲**
"lxml"	pip install lxml	**最快**	優異	網頁很大、要爬很多頁
"html5lib"	pip install html5lib	**最慢**	**最強**	HTML 結構混亂破碎

```python
import os

import pandas as pd
import requests
from bs4 import BeautifulSoup

URL = "https://www.scrapethissite.com/pages/simple/"
HEADERS = {"User-Agent": "..."}

# Step 1：取得網頁 HTML 字串
resp = requests.get(URL, headers=HEADERS, timeout=10)
resp.raise_for_status()
html_text = resp.text

# Step 2：把 HTML 文字轉成 DOM 樹
soup = BeautifulSoup(html_text, "html.parser")
print("網頁標題：", soup.title.text)
```

BeautifulSoup 模組解析網頁

尋找第一個元素

- .select_one(...)：用 CSS selector 找第一個符合的元素。
  若找尋不到時，回傳 None。
  soup.select_one( ) <- 搜尋整個網頁
- "div.country"： 標籤，且 class 含 country。
  ".country-name" : class 含 country-name。
- .prettify( ) : 將找到的元素，轉回 HTML 文字，每一層縮排一格，
  每個標籤獨立一行。
- .get_text(strip=True) : 取出文字且去掉頭尾空白。
  註 : 一律回傳字串，若數字要進行計算，需先轉換。

```
# Step 3：用「第一張」國家卡片，確認選擇器是否正確
first_card = soup.select_one("div.country")
print("\n第一張卡片的 HTML：")
print(first_card.prettify())

# 在第一張國家卡片中找各欄位
name = first_card.select_one(".country-name").get_text(strip=True)
capital = first_card.select_one(".country-capital").get_text(strip=True)
population = first_card.select_one(".country-population").get_text(strip=True)
area = first_card.select_one(".country-area").get_text(strip=True)
print("\n第一筆：", name, capital, population, area)
```

BeautifulSoup 模組解析網頁

尋找所有元素

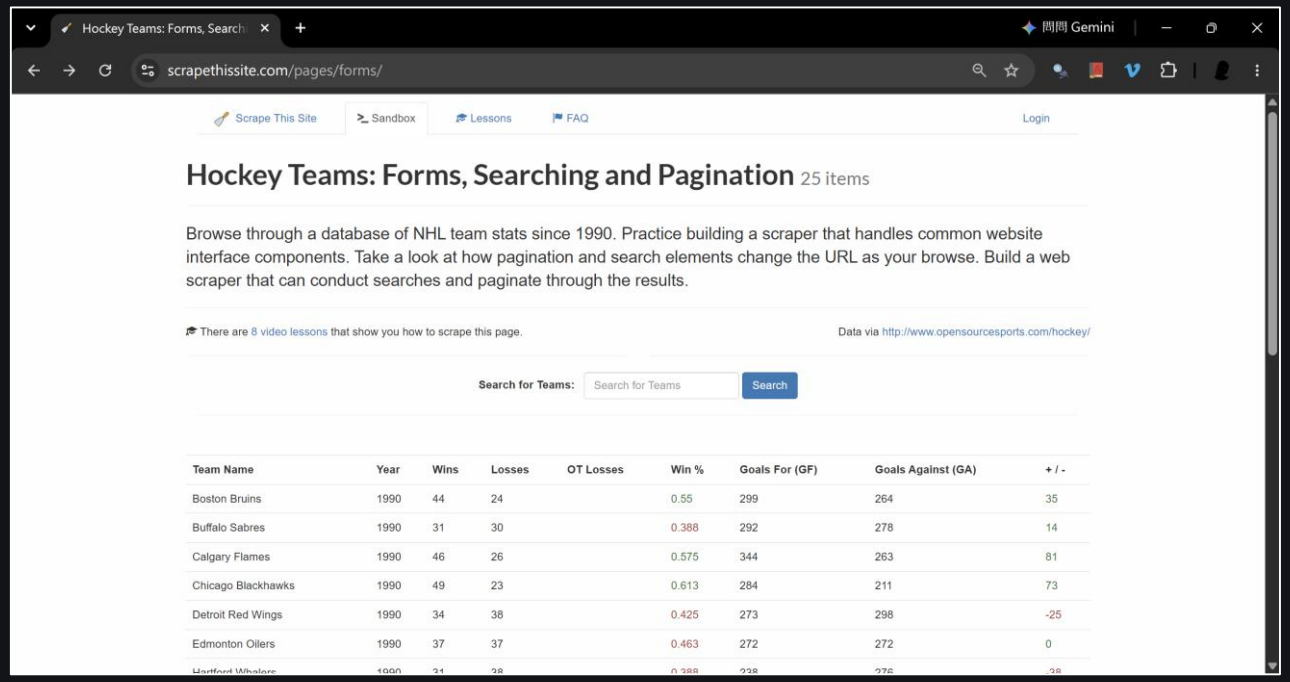
- .select(...)：尋找所有符合的元素。
  若找尋不到時，回傳空集合 [ ]，不是 None。
  註 : 換句話說，找不到時程式不會報錯，可能到分析階段才發現。
- "div.country"： 標籤，且 class 含 country。

```
# Step 4：找所有的國家卡片
cards = soup.select("div.country")
print("\n共找到", len(cards), "張國家卡片")  # 頁面標題寫 250 items
if len(cards) == 0:
    raise RuntimeError("找不到 div.country：請用 DevTools 重新確認網頁結構")

# Step 5：取出每個國家的資訊，且存成一個 dict
countries = []
for card in cards:
    countries.append({
        "country": card.select_one(".country-name").get_text(strip=True),
        "capital": card.select_one(".country-capital").get_text(strip=True),
        "population": int(card.select_one(".country-population").get_text(strip=True)),
        "area_km2": float(card.select_one(".country-area").get_text(strip=True)),
    })

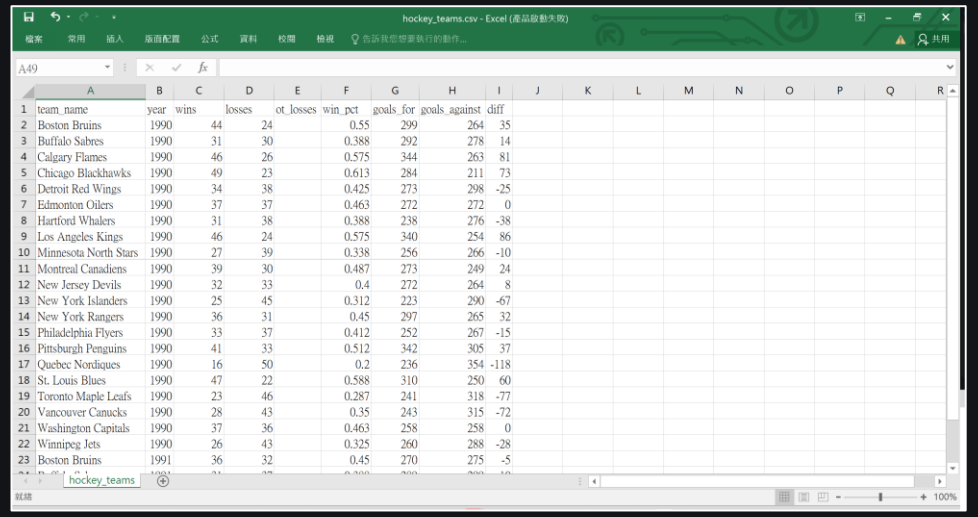
# Step 6：list[dict] → DataFrame → CSV ----------
df = pd.DataFrame(countries)
output_filepath = os.path.join(
        os.path.dirname(os.path.abspath(__file__)), "countries.csv  "
        )
df.to_csv(output_filepath, index=False, encoding="utf-8-sig")
```

課堂練習 - 3



https://www.scrapethissite.com/pages/forms/

- 利用 BeautifulSoup 模組的 select 系列方法，爬取 Hockey Teams Forms 練習網站第一頁的統計表。且將資料表格存成 hockey_teams.csv 檔。
  欄位名稱：team_name、year、wins、losses、ot_losses、win_pct、goals_for、goals_against、diff。



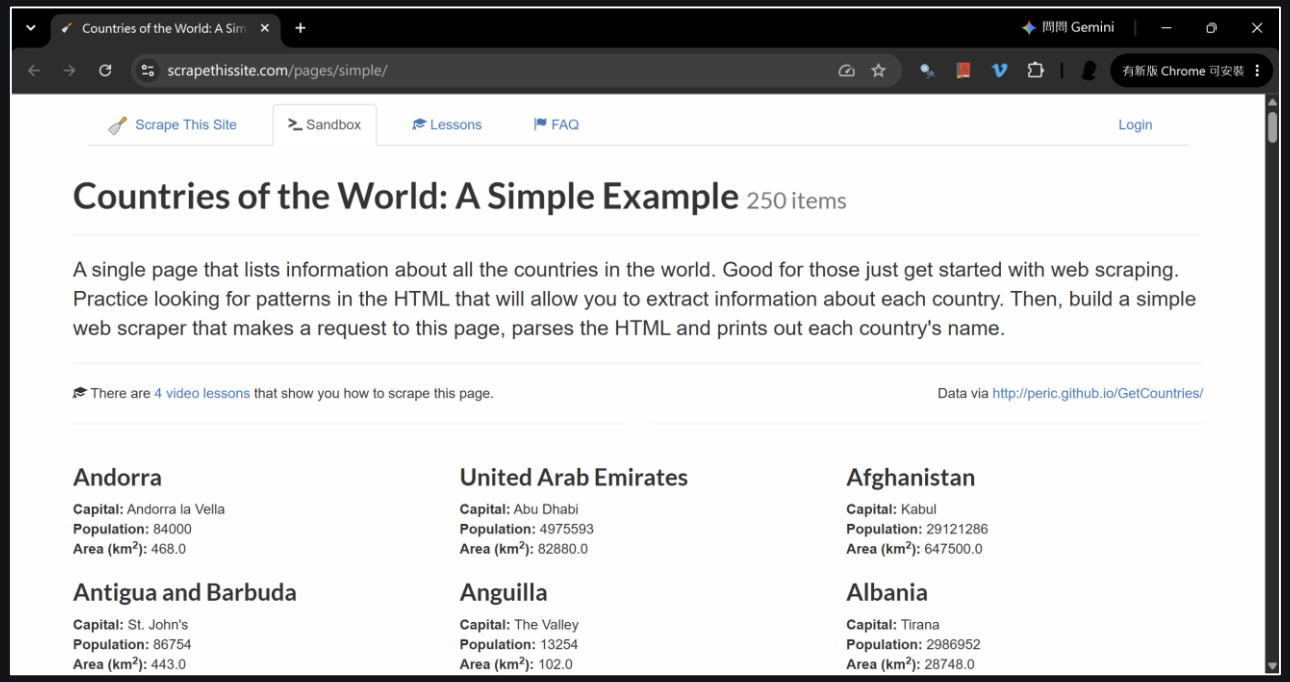
read_html vs BeautifulSoup：何時用哪個

	pd.read_html	BeautifulSoup
適合取什麼資料	只吃 <table>	任何 HTML 結構
寫多少	一行拿到 DataFrame	要自己寫 selector + 迴圈
拿得到屬性嗎	拿不到（只取格子文字）	可以（href、src、class…）
適合	統計表、匯率表、…	卡片、列表、超連結或圖片

BeautifulSoup 找元素的方法

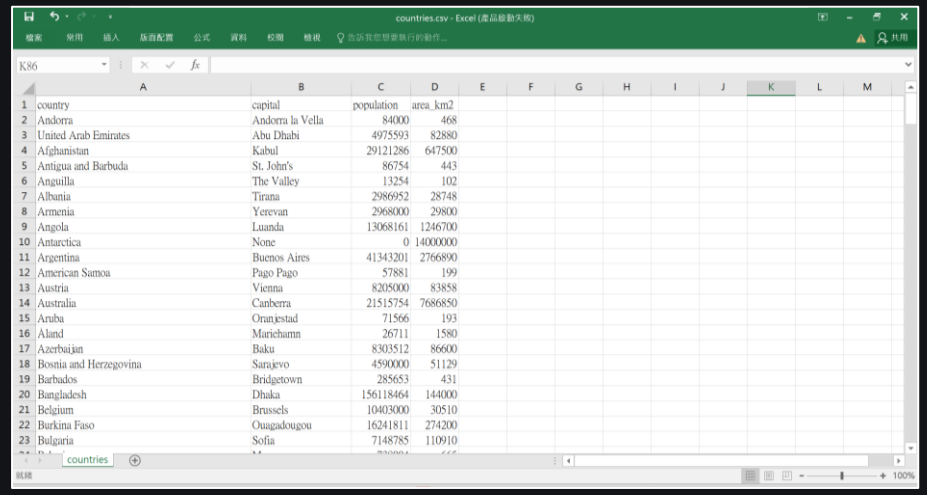
找什麼	find 系列	select 系列
第一個	find(...)	select_one(...)
全部	find_all(...)	select(...)
找不到時	find → None find_all → [ ]	select_one → None select → [ ]
和 DevTools 對照	要自己翻譯成參數 例如 : soup.find_all("div", class_="country")	直接貼上元素面板驗證過的 selector 例如 : soup.select("div.country")
只取前幾筆	find_all(..., limit=3)	要自己切片 select(...)[:3]

課堂練習 - 4



https://www.scrapethissite.com/pages/simple/

- 利用 BeautifulSoup 模組的 find 系列方法，爬取 Countries of the World 網站中所有 250 個國家的資訊。且將資訊存成 countries.csv 檔中。
  欄位名稱 : name、capital、population、area_km2 。



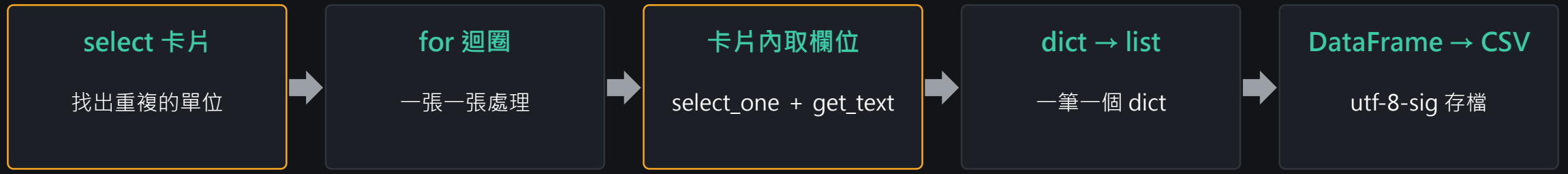
	A	B	C	D
1	country	capital	population	area_km2
2	Andorra	Andorra la Vella	84000	468
3	United Arab Emirates	Abu Dhabi	4975593	82880
4	Afghanistan	Kabul	29121286	647500
5	Antigua and Barbuda	St. John's	86754	443
6	Anguilla	The Valley	13254	102
7	Albania	Tirana	2986952	28748
8	Armenia	Yerevan	2968000	29800
9	Angola	Luanda	13068161	1246700
10	Antarctica	None	0	14000000
11	Argentina	Buenos Aires	41343201	2766890
12	American Samoa	Pago Pago	57881	199
13	Austria	Vienna	8205000	83858
14	Australia	Canberra	21515754	7686850
15	Aruba	Oranjestad	71566	193
16	Åland	Mariehamn	26711	1580
17	Azerbaijan	Baku	8303512	86600
18	Bosnia and Herzegovina	Sarajevo	4590000	51129
19	Barbados	Bridgetown	285653	431
20	Bangladesh	Dhaka	156118464	144000
21	Belgium	Brussels	10403000	30510
22	Burkina Faso	Ouagadougou	16241811	274200
23	Bulgaria	Sofia	7148785	110910

- 請列出人口數前五大的國家。
- 請列出國土面積前五大的國家。

節 3

多筆擷取、存檔與下載

多筆擷取的程式架構


select 卡片
找出重複的單位
for 迴圈
一張一張處理
卡片內取欄位
select_one + get_text
dict → list
一筆一個 dict
DataFrame → CSV
utf-8-sig 存檔

相對路徑 → 完整網址：urljoin

基準網址	相對路徑	urljoin 結果
https://books.toscrape.com/	media/cache/2c/da/2cd….jpg	https://books.toscrape.com/media/cache/2c/da/2cd….jpg
https://books.toscrape.com/	catalogue/page-2.html	https://books.toscrape.com/catalogue/page-2.html

- **URL 結構**：協定、網域、路徑
- **為何不用 +** ../ 與 / 開頭的路徑會拼錯

寫法

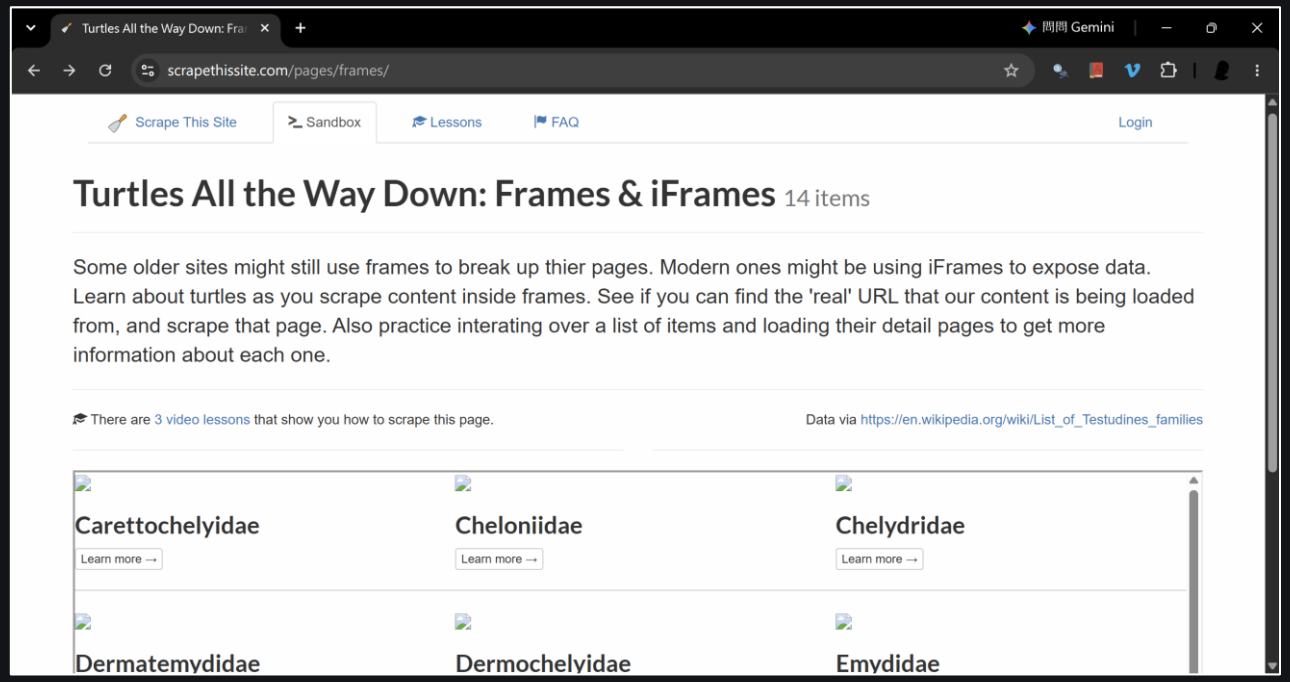
```
from urllib.parse import urljoin

img_url = urljoin(URL, card.select_one("img").get("src"))
```

參考文件：

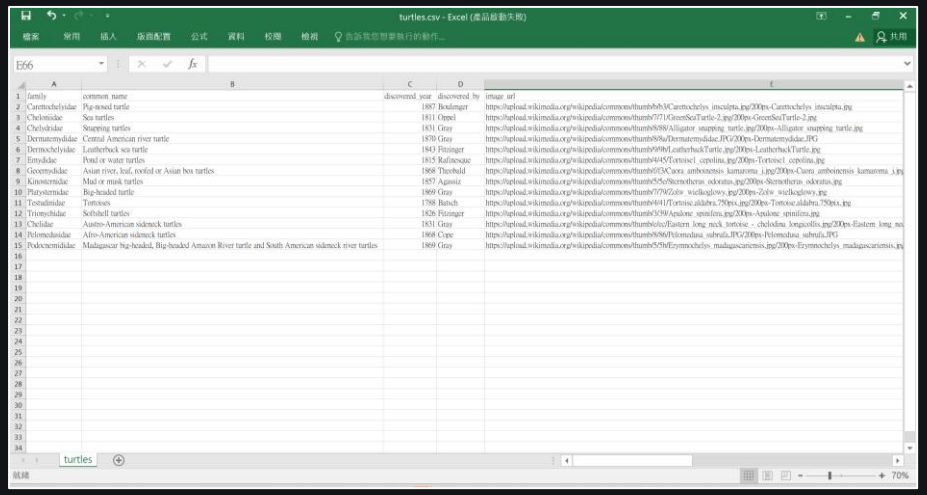
■ Python 官方文件：urllib.parse.urljoin

課堂練習 - 5

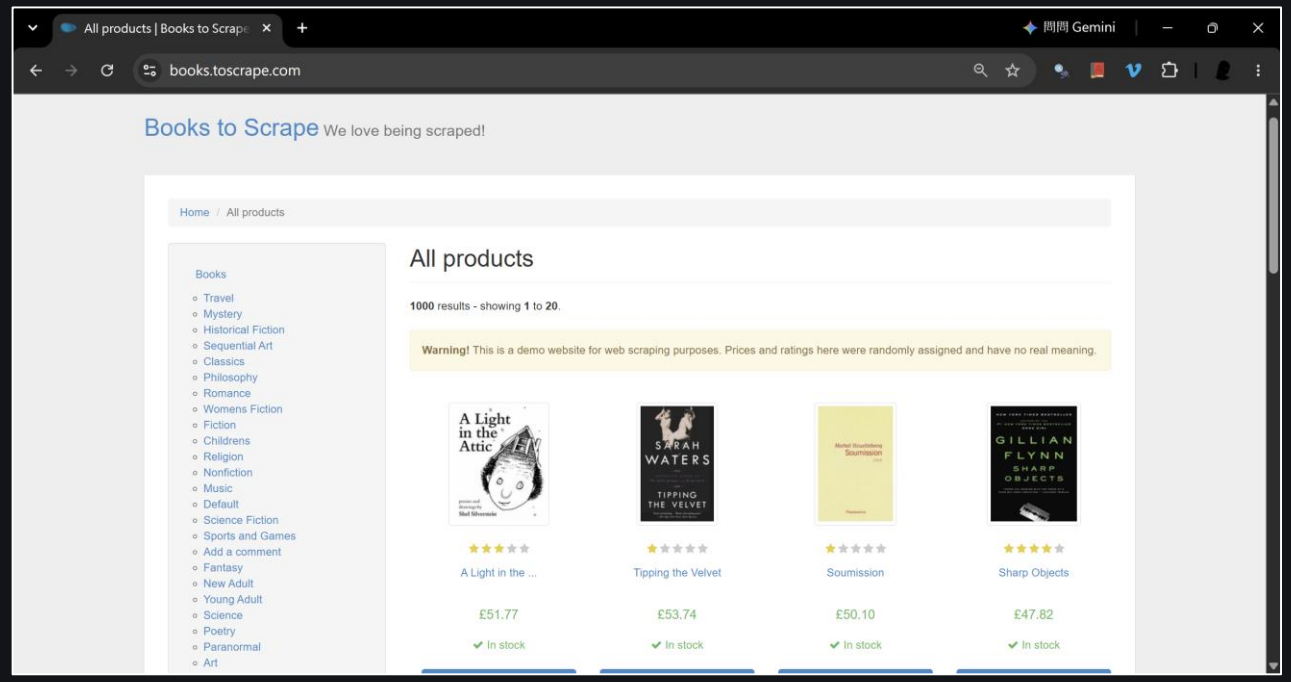


https://www.scrapethissite.com/pages/frames/

- 利用 BeautifulSpup 模組的 select 系列方法，爬取 Turtles All the Way Down 中所有 14 種烏龜的資訊，且將所有資訊存成 turtles.csv 檔。
  欄位名稱 : family、common_name、discovered_year、discovered_by、image_url 。



課堂練習 - 6



https://books.toscrape.com/

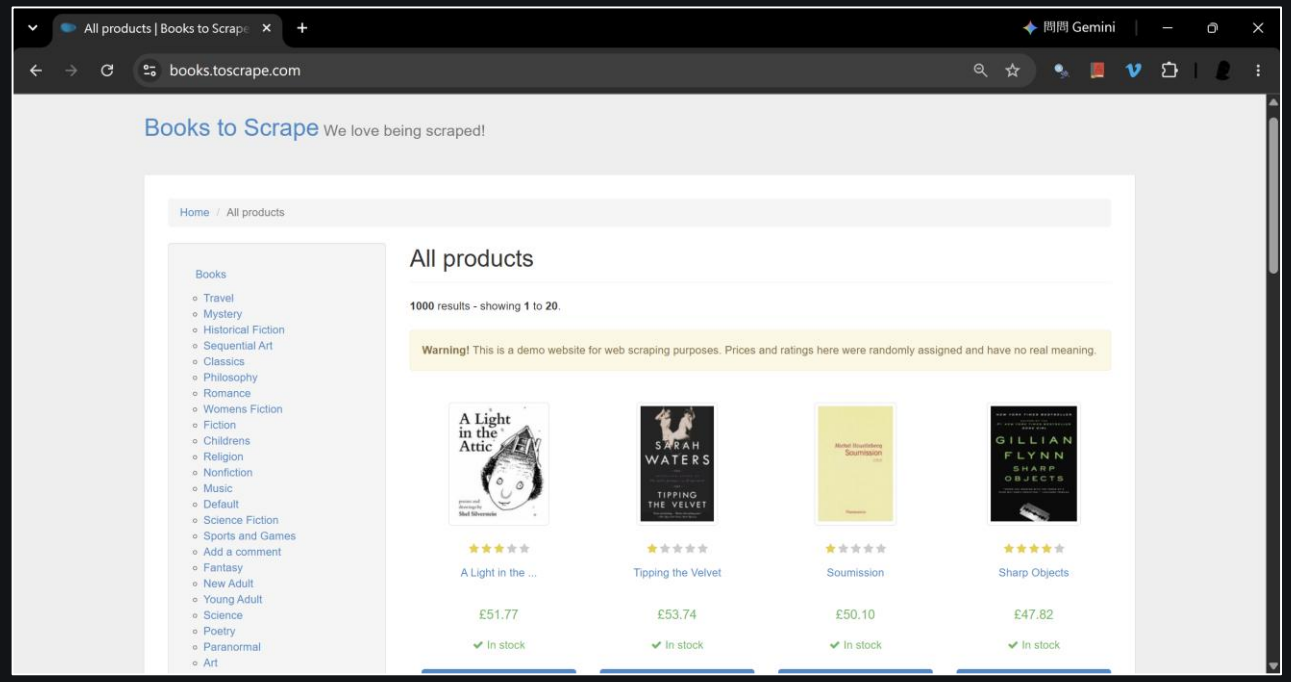
- 利用 BeautifulSoup 模組的 select 系列方法，爬取 Books to Scrape 練習網站首頁前 5 本書的書名。
- 比較畫面上的書名（文字）與完整書名（title 屬性）有何不同？

預期輸出（2026/10 實查）

A Light in the ... → A Light in the Attic
Tipping the Velvet → Tipping the Velvet
Soumission → Soumission
Sharp Objects → Sharp Objects
Sapiens: A Brief History ... → Sapiens: A Brief History of Humankind

概念　看得到的文字不一定是完整資料，屬性也可以是資料。

課堂練習 - 7



https://books.toscrape.com/

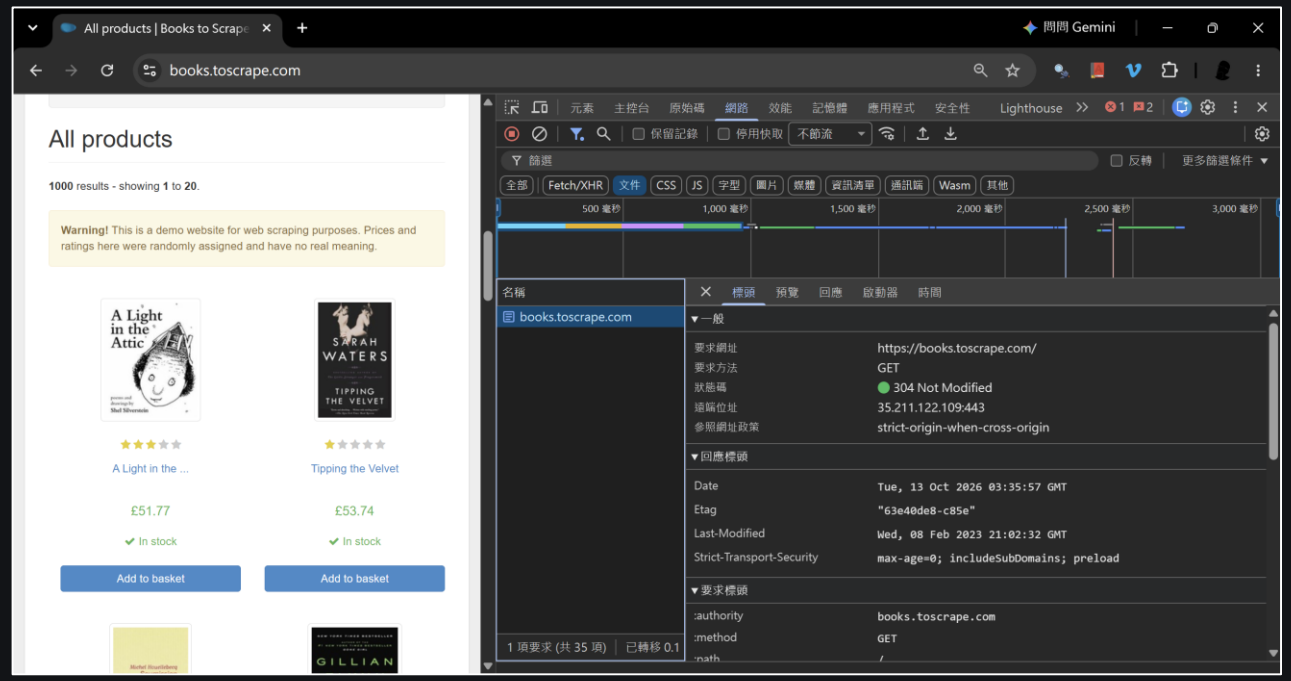
概念　亂碼可能是讀取時用錯編碼。

- 爬取 Books to Scrape 首頁，印出 resp.encoding 與第一本書的價格，找出價格出現 Â£ 亂碼的原因。
- 分別用 resp.encoding 與 resp.content 兩種方法修正亂碼。

預期輸出（2026/10 實查）

```
Content-Type 標頭：text/html
resp.encoding：ISO-8859-1
apparent_encoding：utf-8
[亂碼]  價格 = Â£51.77
[修法1] 價格 = £51.77
[修法2] 價格 = £51.77
```

修正亂碼的兩種做法




```
print(resp.encoding)          # ISO-8859-1
print(resp.apparent_encoding) # utf-8（從內容推測）

# 修法 1：先指定編碼，再取 .text
resp.encoding = "utf-8"  # 或 resp.apparent_encoding
soup = BeautifulSoup(resp.text, "html.parser")

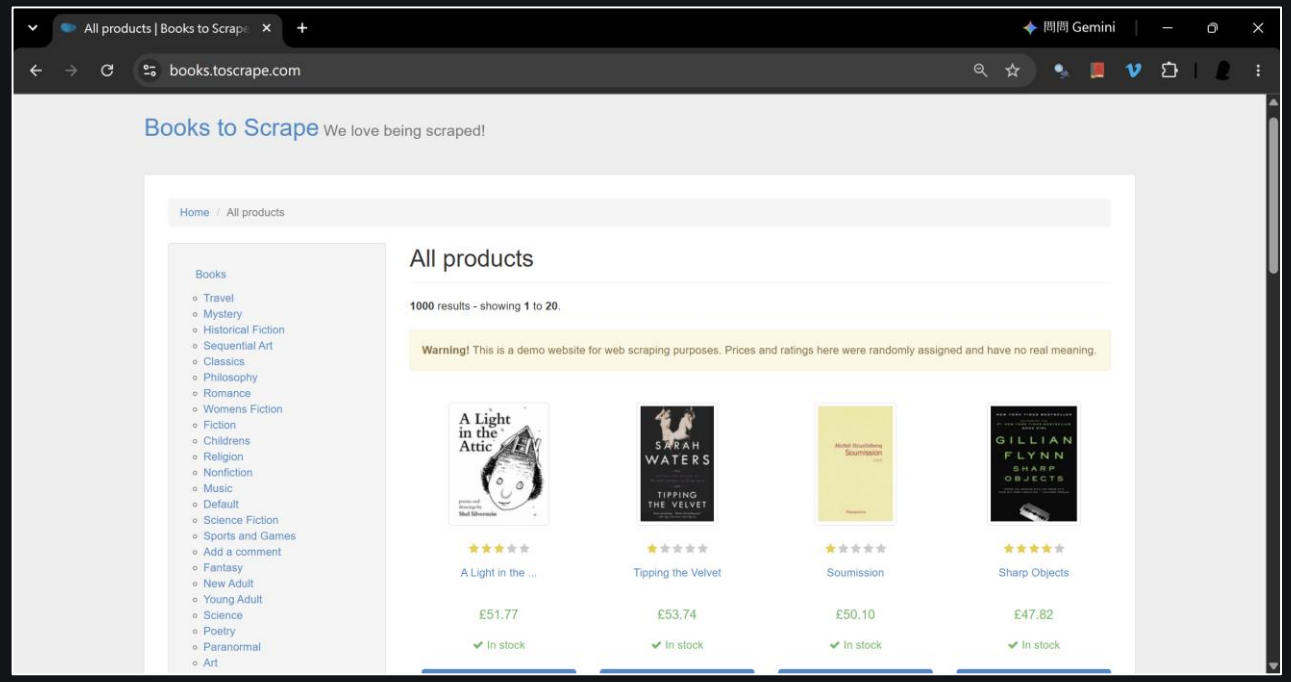
# 修法 2：給 bytes，讓 BeautifulSoup 讀 <meta charset>
soup = BeautifulSoup(resp.content, "html.parser")
```

https://books.toscrape.com/

概念　藉由 resp.encoding 查看解析用的編碼。

requests 會看 HTTP 回應標頭，若回應標頭裡沒有編碼資訊，則會採用 ISO-8859-1 預設解碼。

課堂練習 - 8



https://books.toscrape.com/

- 利用 BeautifulSoup 模組的 select 系列方法，統計 Books to Scrape 首頁 20 本書各星等的數量。
- 用一行 selector 列出所有五星書的完整書名。

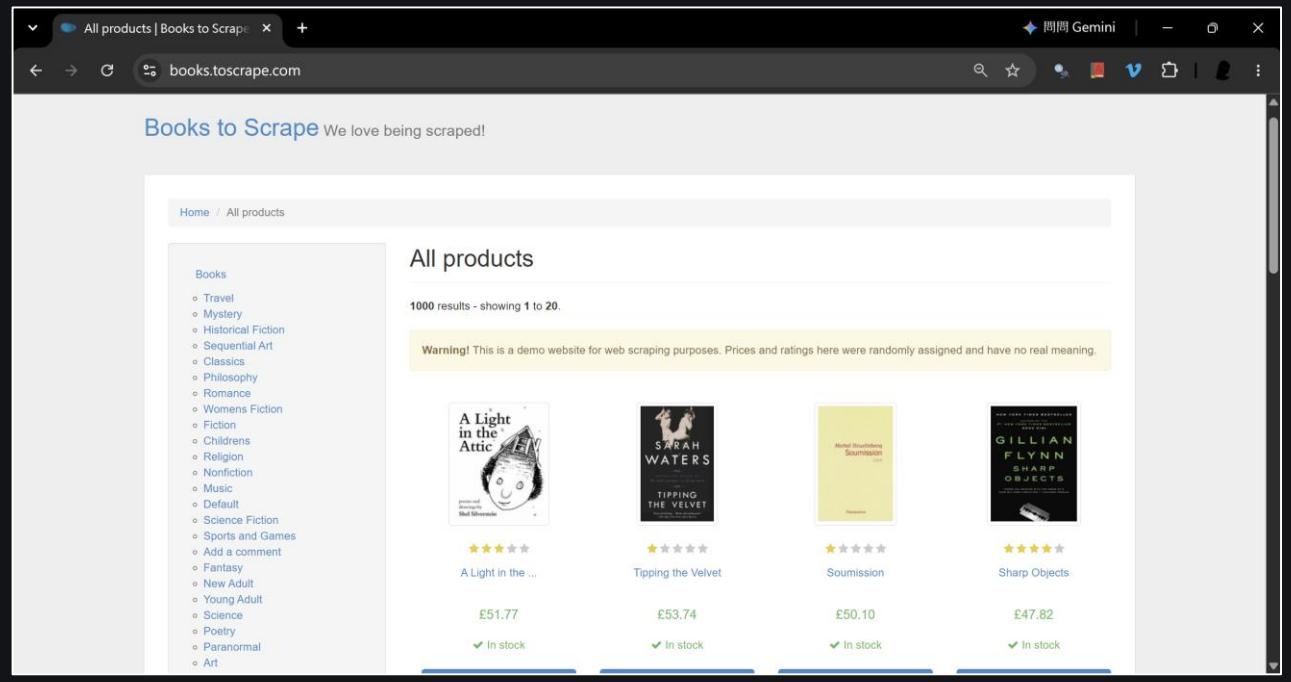
預期輸出（2026/10 實查）

各星等數量：{'One': 6, 'Two': 3, 'Three': 3, 'Four': 4, 'Five': 4} 合計 20 本

五星書共 4 本：
- Sapiens: A Brief History of Humankind
- Set Me Free
- Scott Pilgrim's Precious Little Life (...)
- Rip it Up and Start Again

概念　class 也可以是資料。

課堂練習 - 9



https://books.toscrape.com/

- 爬取 Books to Scrape 首頁左側所有書籍分類的名稱與完整網址，存成 categories.csv 檔。欄位名稱：category、url。
- 進入 Travel 分類，列出該分類所有書名。

預期輸出（2026/10 實查）

共 50 個分類，已存檔：...\categories.csv

Travel 分類共 11 本書（頁面上寫 11 本）：
- It's Only the Himalayas
- Full Moon over Noah’ s Ark: An Odyssey ...
- See America: A Celebration of ...
...（共 11 本）

今天，我們學到了什麼？

下週見 · Python 爬蟲實務應用